1 Training and updating models



2 Training and evaluation data

To train a model, you typically need training data *and* development data for evaluation. What is this evaluation data used for?

- ○ Provide more training examples as a fallback if the training data isn't enough.
- ◉ Check predictions on unseen examples and calculate the accuracy score.
- ○ Define training examples without annotations.

Submit

That's correct! The development data is used to evaluate the model by comparing its predictions on unseen examples to the correct annotations. This is then reflected in the accuracy score.

✓ Completed Next

3 Creating training data (1)

spaCy's rule-based `Matcher` is a great way to quickly create training data for named entity models. A list of sentences is available as the variable `TEXTS`. You can print it to inspect it. We want to find all mentions of different iPhone models, so we can create training data to teach a model to recognize them as `"GADGET"`.

- Write a pattern for two tokens whose lowercase forms match `"iphone"` and `"x"`.
- Write a pattern for two tokens: one token whose lowercase form matches `"iphone"` and a digit.

INPUT

```
import json
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span

with open("exercises/en/iphone.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = Matcher(nlp.vocab)

pattern1 = [{"LOWER": "iphone"}, {"LOWER": "x"}]
pattern2 = [{"LOWER": "iphone"}, {"IS_DIGIT": True}]

matcher.add("GADGET", [pattern1, pattern2])

docs = []
for doc in nlp.pipe(TEXTS):
    matches = matcher(doc)
    spans = [Span(doc, start, end, label=match_id)
             for match_id, start, end in matches]
    print(spans)
    doc.ents = spans
    docs.append(doc)
```

Run code | Submit

OUTPUT

```
[iPhone X]
[iPhone X]
[iPhone X]
[iPhone 8]
[iPhone 11, iPhone 8]
[]
✔ Nice! Now let's use those patterns to quickly bootstrap some training
data for our model.
```

Show hints | Show solution | Reset

✓ Completed | Next

4 Creating training data (2)

After creating the data for our corpus, we need to save it out to a `.spacy` file. The code from the previous example is already available.

- Instantiate the `DocBin` with the list of `docs`.
- Save the `DocBin` to a file called `train.spacy`.

INPUT

```
import json
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span, DocBin

with open("exercises/en/iphone.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = Matcher(nlp.vocab)

pattern1 = [{"LOWER": "iphone"}, {"LOWER": "x"}]
pattern2 = [{"LOWER": "iphone"}, {"IS_DIGIT": True}]

matcher.add("GADGET", [pattern1, pattern2])

docs = []
for doc in nlp.pipe(TEXTS):
    matches = matcher(doc)
    spans = [Span(doc, start, end, label=match_id)
             for match_id, start, end in matches]
    doc.ents = spans
    docs.append(doc)

doc_bin = DocBin(docs=docs)
doc_bin.to_disk("./train.spacy")
```

Run code | Submit

OUTPUT

```
✔ Well done! The pipeline is now ready, so let's start writing the
training loop.
```

Show hints | Show solution | Reset

✓ Completed | Next

6 The training config

The `config.cfg` file is the "single source of truth" for training a pipeline with spaCy. Which of the following is **not true** about the config?

- ◯ It allows you to configure the training process and hyperparameters.
- ◯ It helps make your training more reproducible.
- ◉ It creates an installable Python package with your pipeline.
- ◯ It defines the pipeline's components and their settings.

Submit

That's correct! The config file includes all settings related to training and how to set up the pipeline, but it doesn't package your pipeline. To create an installable Python package, you can use the `spacy package` command.

✓ Completed Next

7 Generating a config file

The `init config` command auto-generates a config file for training with the default settings. We want to train a named entity recognizer, so we'll generate a config file for one pipeline component, `ner`. Because we're executing the command in a Jupyter environment in this course, we're using the prefix `!`. If you're running the command in your local terminal, you can leave this out.

Part 1

- Use spaCy's `init config` command to auto-generate a config for an English pipeline.
- Save the config to a file `config.cfg`.
- Use the `--pipeline` argument to specify one pipeline component, `ner`.

TERMINAL

```
!python -m spacy init config ./config.cfg --lang en --pipeline ner
```

Run command

OUTPUT

```
⚠ To generate a more effective transformer-based config (GPU-only),
install the spacy-transformers package and re-run this command. The config
generated now does not use transformers.
ℹ Generated config template specific for your use case
- Language: en
- Pipeline: ner
- Optimize for: efficiency
- Hardware: CPU
- Transformer: None
✔ Auto-filled config with all values
✔ Saved config
config.cfg
You can now add your data and train your pipeline:
python -m spacy train config.cfg --paths.train ./train.spacy --paths.dev ./dev.s
pacy
```

Show hints | Show solution | Reset

Part 2

Let's take a look at the config spaCy just generated! You can run the command below to print the config to the terminal and inspect it.

TERMINAL

```
!cat ./config.cfg
```

Run command

OUTPUT

```
[paths]
train = null
```

8 Using the training CLI

Let's use the config file generated in the previous exercise and the training corpus we've created to train a named entity recognizer!

The `train` command lets you train a model from a training config file. A file `config_gadget.cfg` is already available in the directory `exercises/en`, as well as a file `train_gadget.spacy` containing the training examples, and a file `dev_gadget.spacy` containing the evaluation examples. Because we're executing the command in a Jupyter environment in this course, we're using the prefix `!`. If you're running the command in your local terminal, you can leave this out.

- Call the `train` command with the file `exercises/en/config_gadget.cfg`.
- Save the trained pipeline to a directory `output`.
- Pass in the `exercises/en/train_gadget.spacy` and `exercises/en/dev_gadget.spacy` paths.

TERMINAL

```
!python -m spacy train ./exercises/en/config_gadget.cfg --output ./output --paths.train
./exercises/en/train_gadget.spacy --paths.dev ./exercises/en/dev_gadget.spacy
```

Run command

OUTPUT

```
✔ Created output directory: output
i Saving to output directory: output
i Using CPU

=========================== Initializing pipeline ===========================
✔ Initialized pipeline

============================= Training pipeline =============================
i Pipeline: ['tok2vec', 'ner']
i Initial learn rate: 0.001
E    #       LOSS TOK2VEC  LOSS NER  ENTS_F  ENTS_P  ENTS_R  SCORE
---  ------  ------------  --------  ------  ------  ------  ------
  0       0          0.00     20.33    1.69    1.04    4.44    0.02
  1     200         44.62   1009.88   70.65   69.15   72.22    0.71
  2     400         67.14    251.76   85.87   84.04   87.78    0.86
  4     600         71.61    111.96   83.62   85.06   82.22    0.84
  6     800         48.18     71.40   83.87   81.25   86.67    0.84
  9    1000         51.99     42.19   83.52   82.61   84.44    0.84
 12    1200         51.45     20.48   84.88   89.02   81.11    0.85
 16    1400         85.83     25.09   82.95   84.88   81.11    0.83
 22    1600        103.87     39.77   86.86   89.41   84.44    0.87
 28    1800        100.95     15.99   90.22   88.30   92.22    0.90
 36    2000         96.23     29.29   87.15   87.64   86.67    0.87
```

Show hints | Show solution | Reset

✓ Completed | Next

9 Exploring the model

Let's see how the model performs on unseen data! To speed things up a little, we already ran a trained pipeline for the label `"GADGET"` over some text. Here are some of the results:

Text	Entities
Apple is slowing down the iPhone 8 and iPhone X - how to stop it	`(iPhone 8, iPhone X)`
I finally understand what the iPhone X 'notch' is for	`(iPhone X,)`
Everything you need to know about the Samsung Galaxy S9	`(Samsung Galaxy,)`
Looking to compare iPad models? Here's how the 2018 lineup stacks up	`(iPad,)`
The iPhone 8 and iPhone 8 Plus are smartphones designed, developed, and marketed by Apple	`(iPhone 8, iPhone 8)`
what is the cheapest ipad, especially ipad pro???	`(ipad, ipad)`
Samsung Galaxy is a series of mobile computing devices designed, manufactured and marketed by Samsung Electronics	`(Samsung Galaxy,)`

Out of all the entities in the texts, **how many did the model get correct**? Keep in mind that incomplete entity spans count as mistakes, too! Tip: Count the number of entities that the model *should* have predicted. Then count the number of entities it *actually* predicted correctly and divide it by the number of total correct entities.

- ◯ 45%
- ◯ 60%
- ◉ 70%
- ◯ 90%

Submit

That's correct! On our test data, the model achieved an accuracy of 70%.

✓ Completed Next

11 Good data vs. bad data

Here's an excerpt from a training set that labels the entity type `TOURIST_DESTINATION` in traveler reviews.

```
doc1 = nlp("i went to amsterdem last year and the canals were beautiful")
doc1.ents = [Span(doc1, 3, 4, label="TOURIST_DESTINATION")]

doc2 = nlp("You should visit Paris once, but the Eiffel Tower is kinda boring")
doc2.ents = [Span(doc2, 3, 4, label="TOURIST_DESTINATION")]

doc3 = nlp("There's also a Paris in Arkansas, lol")
doc3.ents = []

doc4 = nlp("Berlin is perfect for summer holiday: great nightlife and cheap beer!")
doc4.ents = [Span(doc4, 0, 1, label="TOURIST_DESTINATION")]
```

Part 1

Why is this data and label scheme problematic?

- (●) Whether a place is a tourist destination is a subjective judgement and not a definitive category. It will be very difficult for the entity recognizer to learn.
- ( ) Paris should also be labelled as tourist destinations for consistency. Otherwise, the model will be confused.
- ( ) Rare out-of-vocabulary words like the misspelled 'amsterdem' shouldn't be labelled as entities.

Submit

That's correct! A much better approach would be to only label `"GPE"` (geopolitical entity) or `"LOCATION"` and then use a rule-based system to determine whether the entity is a tourist destination in this context. For example, you could resolve the entities types back to a knowledge base or look them up in a travel wiki.

Part 2

- Rewrite the `doc.ents` to only use spans of the label `"GPE"` (cities, states, countries) instead of `"TOURIST_DESTINATION"`.
- Don't forget to add spans for the `"GPE"` entities that weren't labeled in the old data.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.blank("en")

doc1 = nlp("i went to amsterdem last year and the canals were beautiful")
doc1.ents = [Span(doc1, 3, 4, label="GPE")]

doc2 = nlp("You should visit Paris once, but the Eiffel Tower is kinda boring")
doc2.ents = [Span(doc2, 3, 4, label="GPE")]

doc3 = nlp("There's also a Paris in Arkansas, lol")
doc3.ents = [
    Span(doc3, 4, 5, label="GPE"),
    Span(doc3, 6, 7, label="GPE")
]

doc4 = nlp("Berlin is perfect for summer holiday: great nightlife and cheap beer!")
doc4.ents = [Span(doc4, 0, 1, label="GPE")]
```

Run code | Submit

OUTPUT

```
✔ Great work! Once the model achieves good results on detecting GPE
entities in the traveler reviews, you could add a rule-based component to
determine whether the entity is a tourist destination in this context. For
example, you could resolve the entities types back to a knowledge base or look
them up in a travel wiki.
```

Show hints | Show solution | Reset

✓ Completed | Next

12 Training multiple labels

Here's a small sample of a dataset created to train a new entity type `"WEBSITE"`. The original dataset contains a few thousand sentences. In this exercise, you'll be doing the labeling by hand. In real life, you probably want to automate this and use an annotation tool – for example, Brat, a popular open-source solution, or Prodigy, our own annotation tool that integrates with spaCy.

Part 1

- Complete the token offsets for the `"WEBSITE"` entities in the data.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.blank("en")

doc1 = nlp("Reddit partners with Patreon to help creators build communities")
doc1.ents = [
    Span(doc1, 0, 1, label="WEBSITE"),
    Span(doc1, 3, 4, label="WEBSITE"),
]

doc2 = nlp("PewDiePie smashes YouTube record")
doc2.ents = [Span(doc2, 2, 3, label="WEBSITE")]

doc3 = nlp("Reddit founder Alexis Ohanian gave away two Metallica tickets to fans")
doc3.ents = [Span(doc3, 0, 1, label="WEBSITE")]
```

Run code | Submit

OUTPUT

```
✔ Nice work!
```

Show hints | Show solution | Reset

Part 2

A model was trained with the data you just labelled, plus a few thousand similar examples. After training, it's doing great on `"WEBSITE"`, but doesn't recognize `"PERSON"` anymore. Why could this be happening?

- ◯ It's very difficult for the model to learn about different categories like `"PERSON"` and `"WEBSITE"`.
- ◉ The training data included no examples of `"PERSON"`, so the model learned that this label is incorrect.
- ◯ The hyperparameters need to be retuned so that both entity types can be recognized.

Submit

That's correct! If `"PERSON"` entities occur in the training data but aren't labelled, the model will learn that they shouldn't be predicted. Similarly, if an existing entity type isn't present in the training data, the model may "forget" and stop predicting it.

Part 3

- Update the training data to include annotations for the `"PERSON"` entities "PewDiePie" and "Alexis Ohanian".

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.blank("en")

doc1 = nlp("Reddit partners with Patreon to help creators build communities")
doc1.ents = [
    Span(doc1, 0, 1, label="WEBSITE"),
    Span(doc1, 3, 4, label="WEBSITE"),
]

doc2 = nlp("PewDiePie smashes YouTube record")
doc2.ents = [
    Span(doc2, 0, 1, label="PERSON"),
    Span(doc2, 2, 3, label="WEBSITE")
]

doc3 = nlp("Reddit founder Alexis Ohanian gave away two Metallica tickets to fans")
doc3.ents = [
    Span(doc3, 0, 1, label="WEBSITE"),
    Span(doc3, 2, 4, label="PERSON")
]
```

Run code Submit

OUTPUT

```
✔ Good job! After including both examples of the new WEBSITE entities,
as well as existing entity types like PERSON, the model now performs much
better.
```

Show hints Show solution Reset

Chapter 4: Training a neural network model

In this chapter, you'll learn how to update spaCy's statistical models to customize them for your use case – for example, to predict a new entity type in online comments. You'll train your own model from scratch, and understand the basics of how training works, along with tips and tricks that can make your custom NLP projects more successful.

1 Training and updating models 

2 Training and evaluation data

3 Creating training data (1)

4 Creating training data (2)

5 Configuring and running the training 

6 The training config

7 Generating a config file

8 Using the training CLI

9 Exploring the model

10 Training best practices 

11 Good data vs. bad data

12 Training multiple labels

13 Wrapping up 

« Previous Chapter